



Centre for Credible
Artificial Intelligence

From End-to-end to Step-by-step: Learning to Abstract via Abductive Reinforcement Learning

Maciej Świechowski

CCAI Monday Seminar, 15.06.2026

Voting results

From End-to-end to Step-by-step: Learning to Abstract via Abductive Reinforcement Learning [IJCAI 2025] **6 votes**

Wang, Z., Wang, J., Chen, X., Wang, M., Ma, M., Wang, Z., Zhou, Z., Yang, T. and Dai, W.Z.

SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement [ICLR 2025] **2 votes**

Navigates Like Me: Understanding How People Evaluate Human-Like AI in Video Games [CHI 2023] **5 votes**

Reinforcement Learning

- **Learning by interacting in a dynamic environment**
- **Maximizing the cumulative reward over certain (or infinite) horizon**
 - negative rewards (penalties) are used too, rarely
- **Continuous feedback loop**

One of the original three types of machine learning:

Reinforcement
Learning

Supervised Learning

Unsupervised
Learning

Self-Supervised
Learning

Semi-Supervised
Learning

...

Markov Decision Process (MDP)

MDP (or its generalizations) **is a formal mathematical framework behind RL**

Richard Bellman, *A Markovian Decision Process*, *Journal of Mathematics and Mechanics*, 1957

Agent: decision maker

$$\langle S, A, P, R, \gamma \rangle$$

S : state space (the set of possible states of environment)

A: action space (the set of actions Agent can make)

P: $S \times A \times S \rightarrow [0, 1]$ - transition probability function. Call notation: $P(s' | s, a)$

R: $S \times A \rightarrow \mathbb{R}$ - reward function. Call notation: $R(s, a, s')$

$\gamma \in [0, 1]$: discount factor, the higher the more future rewards are included in the goal

Reinforcement Learning in practice

Learning:

- Value function: $V(s)$ how “good” state **s** is for the agent
- Action-Value function: $Q(s, a)$ how “good” making action **a** in state **s** is
- Policy function: $\pi(a|s)$ directly which action **a** to make in state **s**

or a combination of the above

Many properties (families) of RL approaches, e.g.:

- model-based vs. model-free
- on-policy vs. off-policy

Motivation for Abstraction

Tasks decomposition

Reusability

- Learning compositional structures
- Example: an agent finds a key and opens the door. With traditional end-to-end RL it cannot easily reuse that “key-finding” ability

Potentially more accurate credit assignment if rewards are delayed or sparse

- Example: an agent gets a reward after 1000-step task. Difficult to attribute what actually caused the success

Better explainability

- Explaining strategies rather than long sequences of actions
- Natural summarizations

Abductive reasoning

Scientific theories are evaluated on the basis of how well they would explain the available evidence

Hypotheses can be updated once we have more evidence

Inference to the Best Explanation

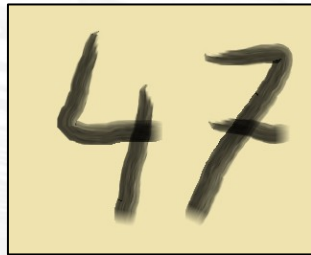
Common-sense reasoning

Less formalized than deductive or inductive reasoning

Abductive learning

(specifically as defined in preliminaries in the paper)

1. Hybrid neuro-symbolic (NeSy) framework combining machine learning and logical reasoning
2. Neural network processes raw environmental input and outputs primitive facts/labels:



0.95 :: "47"

3. Logical reasoning engine takes those facts and checks them against Knowledge Base

4. Abduction:

- **Correcting NN output:** feedback for retraining
- **Correcting KB:** for example, inducing missing rules

5. Optimization based on the output from 4.

First novelty claimed in the paper

Modelling ML models as dyadic predicates representing atomic procedures for constructing complex procedures for RL

$$\pi^*(a, b) \leftarrow \pi_{0 \rightarrow 1}(a, c) \wedge \pi_{1 \rightarrow 2}(c, b)$$

Applying policy π^* to transition from state a to state b can be divided into two steps as above

Open-universe language

- more about this later

Abstract State Machine (ASM)

Defined over the base MDP as:

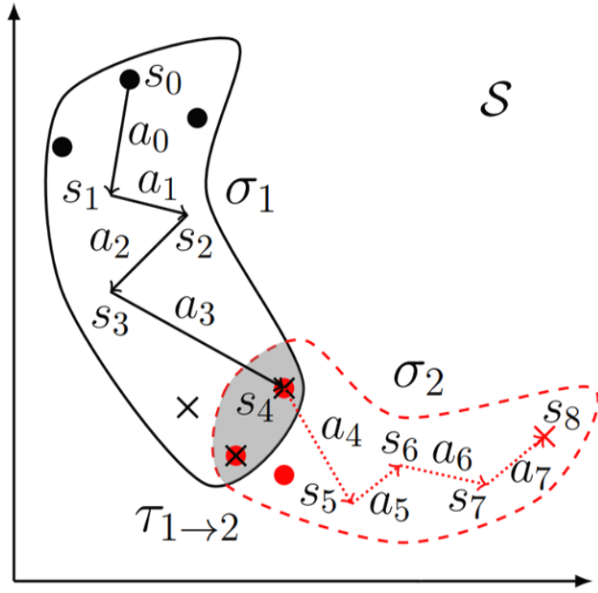
$$\Delta = \langle \Sigma, T, P, R', \gamma \rangle$$

$\Sigma = \{\sigma \mid \sigma \subseteq S\}$ is a countable set of abstract states

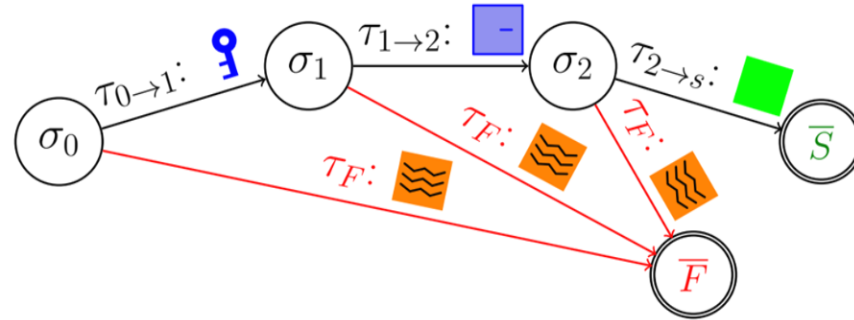
$T = \{\tau_{i \rightarrow j} \mid \tau_{i \rightarrow j} = t(\sigma_i) \cap s(\sigma_j)\}$ is a set of abstract state transitions,
where $s(\sigma)$ and $t(\sigma)$ are subsets of initial and terminate states of σ , respectively

$R' = \{R\} \cup \{R_0^\sigma, R_1^\sigma, \dots\}$ is a set of augmented reward functions;
each one is active in i -th abstract state

ASM illustration



(a)



$$\begin{aligned}
 \text{Success}(a, b) &\leftarrow \pi_{0 \rightarrow 1}(a, c) \wedge c \in \tau_{0 \rightarrow 1} \\
 &\quad \wedge \pi_{1 \rightarrow 2}(c, d) \wedge d \in \tau_{1 \rightarrow 2} \\
 &\quad \wedge \pi_{2 \rightarrow s}(d, b) \wedge b \in \bar{S} \\
 F(a, b) &\leftarrow \pi_F(a, b) \wedge b \in \bar{F}
 \end{aligned}$$

(b)

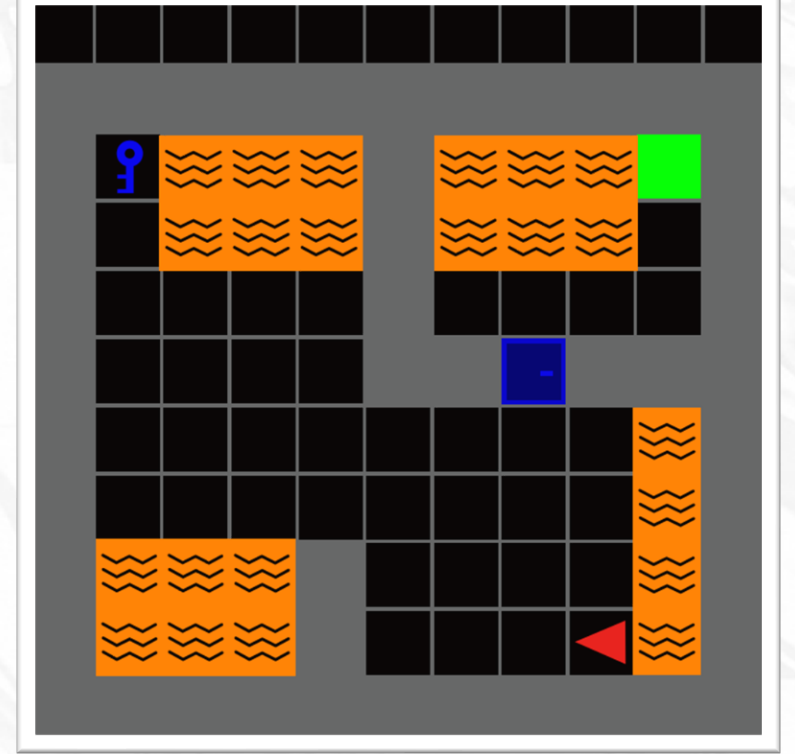


Figure 3: (a) A *ground* trace in $\mathcal{S}$ segmented into two *abstract* steps, illustrated as solid and dotted paths; $\bullet/\times$ in each colour denote the initial/terminate ground states in each step. (b) The induced ASM from the example in Figure 1 with its logic form.

Sub-MDP for an abstract state σ_i

$$\mathcal{M}_i^\sigma = \langle \sigma_i, A, P, R_i^\sigma, \gamma \rangle$$

for which the objective is to learn Atomic Policy functions $\pi: \sigma_i \rightarrow A$ that navigates the agent from $s(\sigma_i)$ to $t(\sigma_i)$

If σ_i (σ_j in paper) serves as a terminal state, then $\tau_{* \rightarrow i} = s(\sigma_i) = t(\sigma_i)$

Augmented reward function:

$$R_i^\sigma(s, a, s') = \begin{cases} \theta_i & \text{if } s' \in t(\sigma_i) \\ R(s, a, s') & \text{otherwise} \end{cases}$$

Algebraic perspective

Sub-MDP forms a **subalgebra** of the original MDP.

- subset that is closed under all the operations of that structure

In theory, this definition enables recursive division of sub-MDPs until it is trivial to solve.

Abductive Abstract RL (A2RL)

The authors define A2RL for a given a ground MDP $\langle S, A, P, R, \gamma \rangle$ as the procedure:

- I. Discover a set of abstract states $\Sigma = \{\sigma_0, \dots\}$ and construct ASM $\Delta = \langle \Sigma, T, P, R', \gamma \rangle$
- II. Learn a set of state abstraction functions Φ , where each $\phi_i: S \rightarrow \{T, F\}$ such that $\phi_i(s) = T$ iff $s \in \sigma_i$
- III. Induce the abstract transition model T
- IV. Solve the sub-MDP derived from Δ by training atomic policy $\pi_{i \rightarrow j}$ for each state transition $\tau_{i \rightarrow j}$

Object Discovery

“Let’s rethink the question in [Russell, 2015]: can machine learning infer the existence of objects from raw data that contain no explicit object references?”

The authors argue that A2RL tries to answer it positively.

Object: an abstract state transition function that generalizes across domains.

“In other words, the function of key is the most important thing for this concept, since it helps the agent change from a blocked state to unblocked, no matter it is a in Minigrid, a bomb in The Legend of Zelda, or a rune stone in Tomb Raider”



Centre for Credible
Artificial Intelligence

Method Implementation

(Section 4 in the paper)

Dual control loop

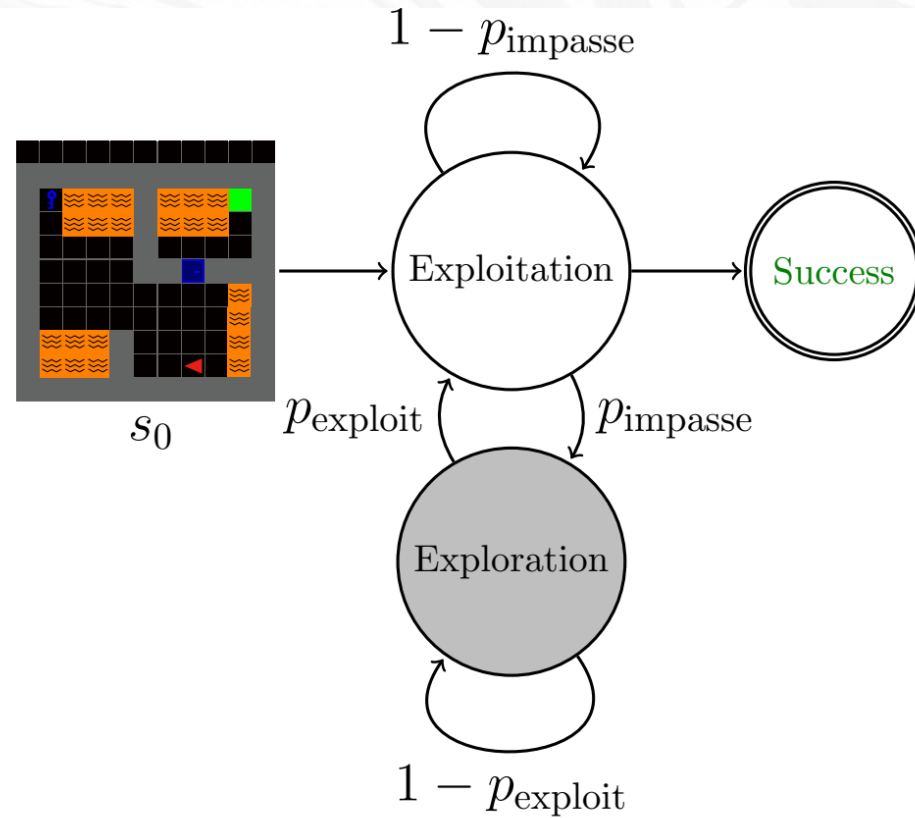


Figure 4: The impasse-driven exploration-exploitation of A2RL

The agent shifts between standard RL exploration and rule-based structural exploitation

Exploitation

Given an ASM $\Delta = \langle \Sigma, T, P, R', \gamma \rangle$ and an input ground state $\mathbf{s}_t$

- the state abstraction function Φ is called to identify abstract state σ_i

A2RL uses T to calculate abstract plan Π^* - a path from σ_i to target (next) abstract state.

The plan Π^* is executed step-by-step by calling the atomic policies $\pi_{i \rightarrow j}$ in this path.

As I understand this:

- **exploitation** assumes that the current abstraction is correct and already built
- **if not:** the algorithm switches to exploration, which modifies the abstraction itself

Exploitation: example

NoKey → HasKey → IsNearDoor → DoorOpened → GoalReached

The ground MDP has many calls to sub-symbolic models and image observations.
The ASM contains only high-level stages.

The exploitation phase operates on this compact structure.

Exploitation: impasse

Uses **Drift Diffusion Model (DDM)**:

$$dx = v \cdot dt + \sigma \cdot dW_t$$

x: as the agent operates without reward, evidence accumulates.

Drift Rate (v) and Noise (σ): control acceleration speed.

dW_t models the standard Brownian motion.

The impasse probability triggers the transition to exploration:

$$p_{\text{impasse}} \propto x.$$

Exploration

- **The last atomic state from exploitation is the start state for exploration**
- **Use random policy from action space A**
 - If the agent fails: restart the episode
- **Compute probabilities of the current state being in a known abstract state and accumulate for each of them:**

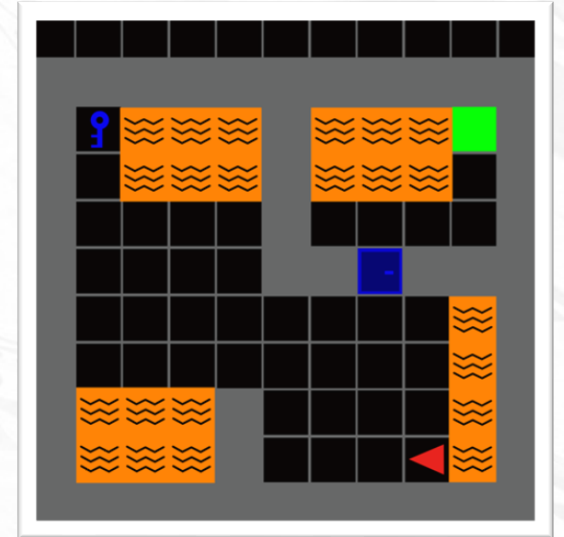
$$x = \max_i P(\Phi(s) = i)$$

- **Update the probability of switching:**

$$P_{\text{exploit}} \propto x$$

At the end of exploration

- Raw trajectory data surrounding the impasse is analyzed and each trajectory is segmented into the **exploitation** (first) and **exploration** (later) parts
- The exploitation part is further divided based on abstract states
- All exploration segments are clustered using [raw images, neighboring abstract states] to form new abstract state σ_{new} per cluster



(some ground states around the impasse are taken away from previous abstract state)

At the end of exploration

- **Update of logical rules** using fixed domain First-Order Logic knowledge base
- State abstraction function ϕ_{new} is trained using the visited ground states

- **Update of ASM structure:**

$$\Sigma \leftarrow \Sigma \cup \{\sigma_{\text{new}}\}$$

$$T \leftarrow T \cup \{\tau_{\text{new} \rightarrow \text{old}}\}$$

$$\Phi \leftarrow \Phi \cup \{\phi_{\text{new}}\}$$

$$R' \leftarrow R' \cup \{R_{\text{new}}^{\sigma}\}$$

- **Retraining** policy models for affected sub-MDPs



Centre for Credible
Artificial Intelligence

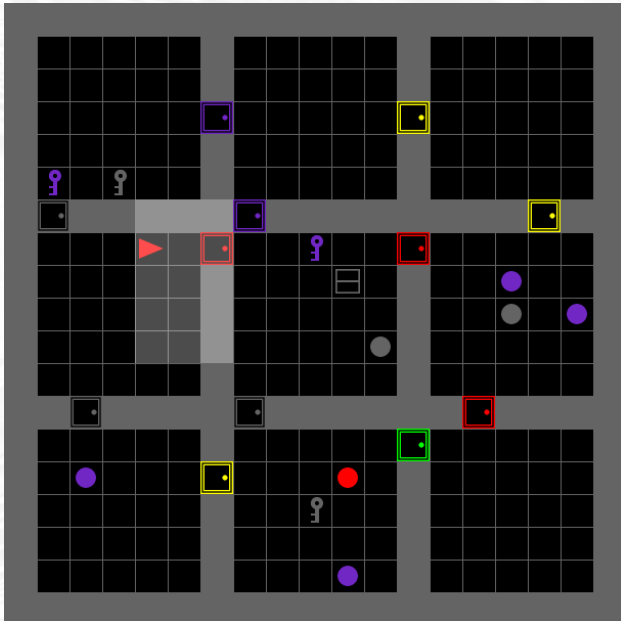
Method Evaluation

(Section 5 in the paper)

Experimental Environment 1

Minigrid

- partially observable highly configurable 2D grid environments of different sizes
- solving maze-like maps, interacting with various objects, avoiding obstacles

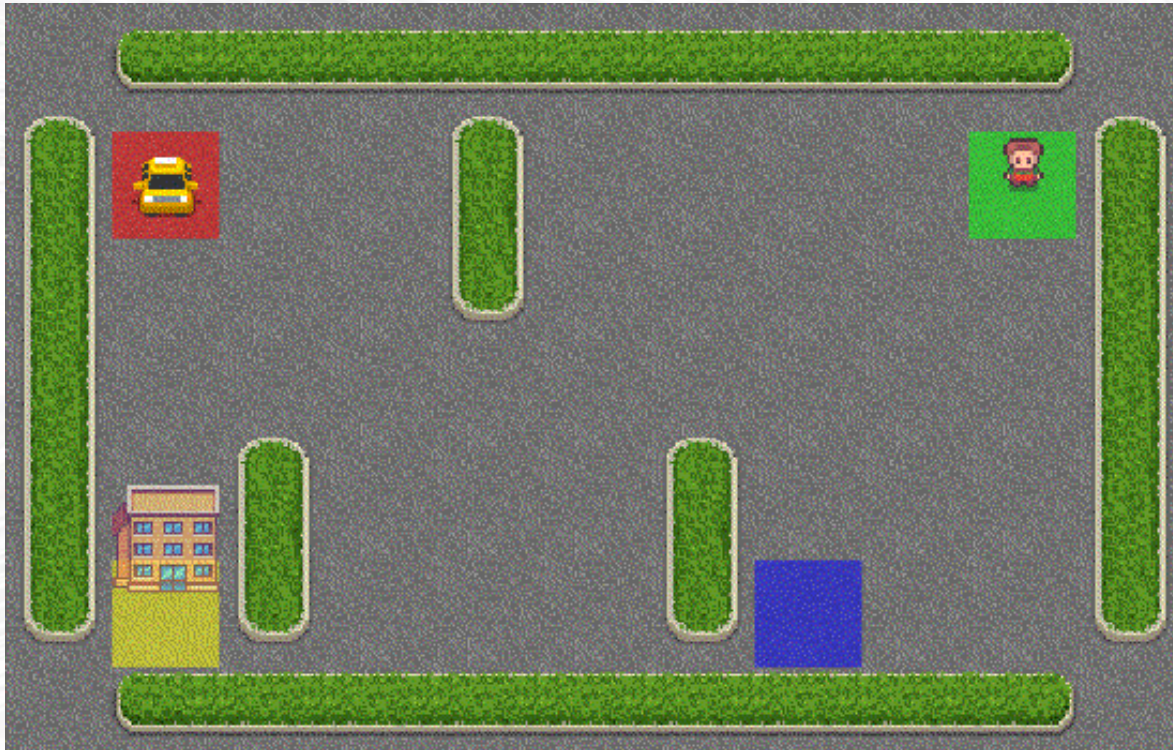


Chevalier-Boisvert et al., *Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks*. In Advances in Neural Information Processing Systems, volume 36, New Orleans, LA, 2023.

Experimental Environment 2

Taxi

- the taxi must pick up the passenger, drive to the destination, and drop them off
- raw pixels as input



Thomas G. Dietterich. ***Hierarchical Reinforcement Learning with the MAXQ Value Function decomposition***. Journal of Artificial Intelligence Research, 13(1):227–303, 2000

RL methods used: PPO2

A modification of Proximal Policy Optimization (PPO), which constraints policy changes to maintain stability

- Uses importance sampling
- Model-free and off-policy gradient method
- Learns a policy directly
- Works for both continuous and discrete action spaces
- Both PPO and PPO2 use the so-called Advantage Estimation method.
 - Advantage: was a specific action better or worse than the policy typically expects?

RL methods used: D3QN

Improves upon DQN (Q-Learning with Deep Neural Networks):

- **Double DQN** (a network for action selection and another one for action evaluation)
- **Dueling Network Architecture** – instead of directly learning $Q(s, a)$, learns $V(s)$ and advantage $A(s, a)$ and combines them:

$$Q(s, a) = V(s) + (A(s, a) - \text{mean}(A))$$

- Model-free and off-policy method
- Works for discrete action spaces

Baselines

Ground Truth:

- handcrafted to set a performance upper-bound

RUDDER

- typical RL approach for tackling delayed reward problems
- implemented on top of the PPO method and only compared with PPO

Arjona-Medina, Jose A., Michael Gillhofer, Michael Widrich, Thomas Unterthiner, Johannes Brandstetter, and Sepp Hochreiter. ***Rudder: Return decomposition for delayed rewards.*** Advances in Neural Information Processing Systems 32 (2019).

Empirical results: Minigrid

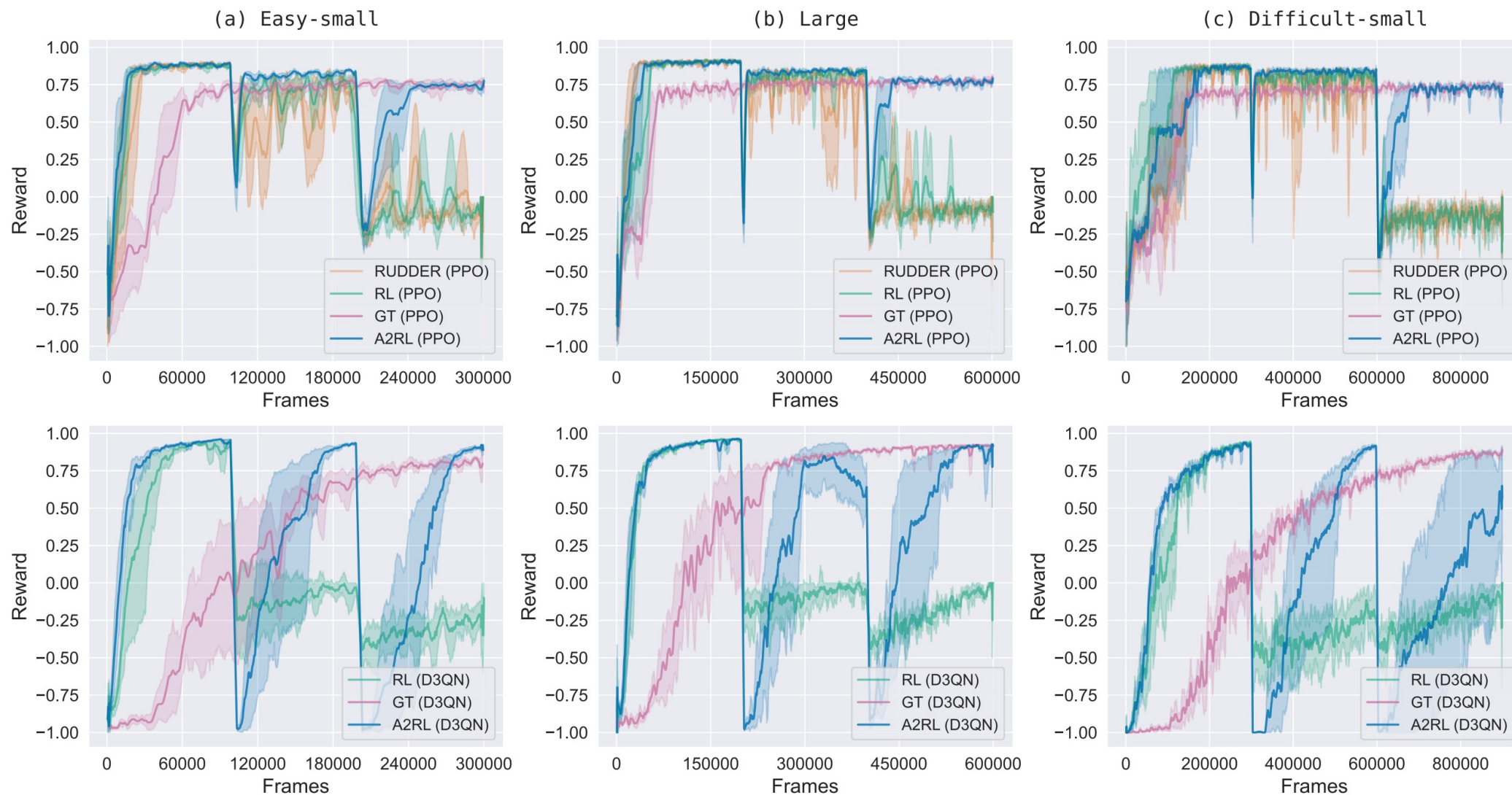
One training with three maps of increasing complexity

- Map1: 300K training steps
- Map2: 600K training steps
- Map3: 900K training steps

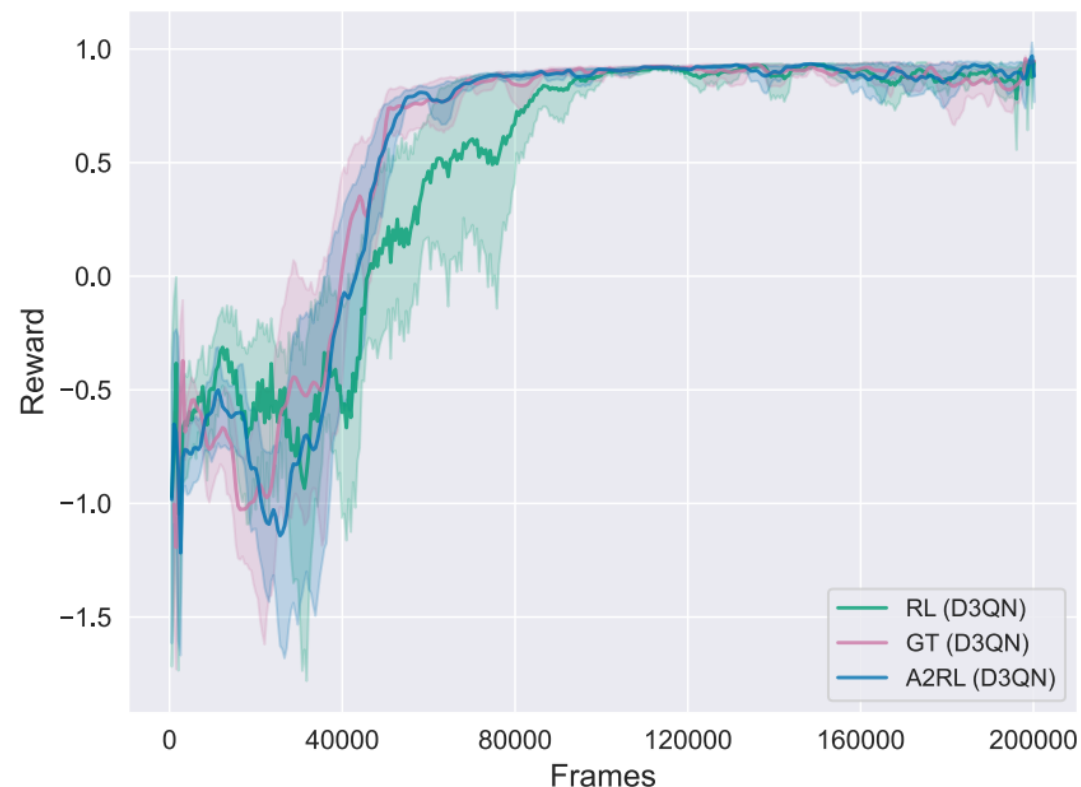
Metric:

- **10-window moving average** applied to **average batch rewards** from the environment
- **shades areas: 95% confidence** (not specified according to which statistical test)

Empirical results: Minigrid



Empirical results: Taxi



Out-of-distribution Generalization

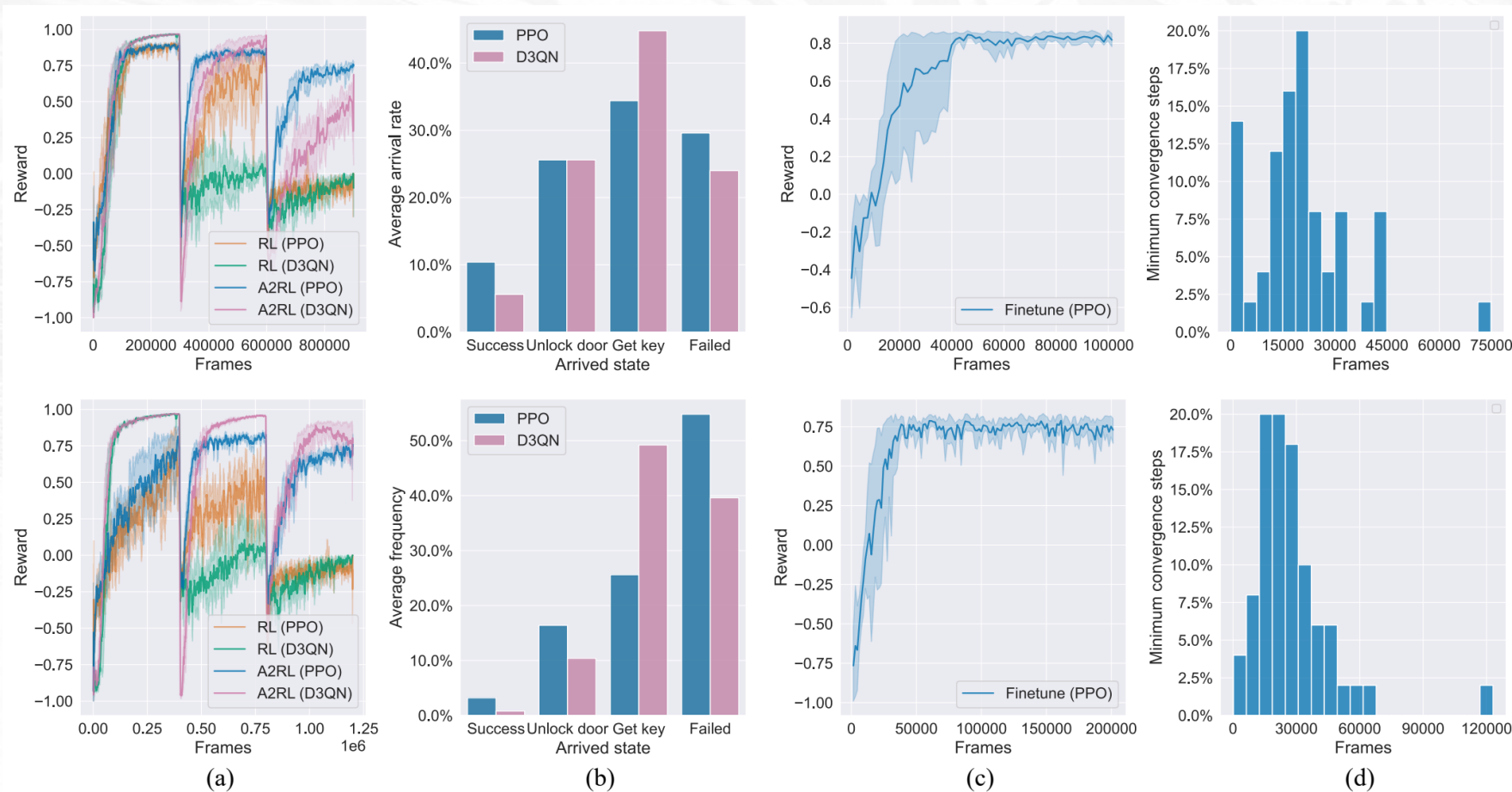


Figure 7: The two rows represent Minigrid-small and Minigrid-large with random maps. (a) The reward curves of A2RL are trained directly on 5 types of random maps. (b) The results of testing on 50 unseen maps after training on random maps, showing the proportion of sub-tasks completed on the 50 unfamiliar maps. For each of the 5 trained models, three tests were conducted on each map, and the best result among them was recorded. (c) The reward curve of A2RL during fine-tuning for generalization on the 50 unseen maps. (d) The distribution of the number of steps required for the reward curve to first achieve convergence during fine-tuning (convergence is defined as achieving success in more than 50 out of the most recent 100 episodes).

Authors Conclusions

A novel framework for decomposition + RL

Abstractions directly from sub-symbolic environments w/o predefined symbols

Future work:

- enhancing the abstraction mechanisms
- expanding application to a wider range of real-world scenarios

“Through the continued development of A2RL, we aim to further unify symbolic reasoning with learning, advancing the creation of more reliable and efficient AI systems capable of addressing complex and unstructured challenges”

XAI idea (just food for thought)

Finding the hidden explanation that makes observations coherent.

Given a set of successful trajectories:

$$(s_1, a_1, s_2, a_2, s_3, a_3, \dots, s_n, a_n)$$

find latent abstract states

$$(\sigma_1, \sigma_2, \sigma_3, \dots, \sigma_m)$$

that explain why the trajectory succeeds.

My idea in this topic

Maintain logical hypotheses about the environment

- prepared manually (the ones want to check)
- formulated automatically on-the-go according to some meta-strategy or by a dedicated ML model

Incorporate rewards related to checking those hypotheses “well enough”

- with certain confidence

Additionally, make use of these hypotheses, at least in the exploration aspect

- for example, do not explore what we hypothesized is fruitless

Previous idea conceptually

Instead of optimizing:

$$\pi(a_t | s_t)$$

with objective

$$\mathbb{E} \left[\sum_t R_{\text{env}} + \lambda * R_{\text{hypothesis}}(h_t) \right]$$

optimize something like:

$$\pi(a_t, h_t | s_t)$$

Any Questions?

Repository related to this paper:

<https://github.com/sporeking/A2RL>

[This] Wang, Zilong, et al. ***From End-to-end to Step-by-step: Learning to Abstract via Abductive Reinforcement Learning***. Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25. 2025. <https://www.ijcai.org/proceedings/2025/0725.pdf>

[Russell, 2015] Stuart Russell. ***Unifying logic and probability***. Communications of the ACM, 58(7):88–97, 2015.